Instituicao de Ensino Demo

Curso: Engenharia de Software

5o Semestre

Material de Estudo

DevOps e Cloud Computing

---

1. INTRODUCAO

DevOps e Cloud Computing sao pilares da engenharia de software moderna. DevOps une desenvolvimento e operacoes para entregar software de forma rapida, confiavel e automatizada. Cloud Computing permite escalar aplicacoes globalmente sem gerenciar infraestrutura fisica. Juntos, esses conceitos transformaram a forma como software e construido, testado e implantado, sendo habilidades essenciais para qualquer engenheiro de software no mercado atual.

Ementa: Cultura DevOps e integracao continua. Containers e Docker. Orquestracao com Kubernetes. Infraestrutura como codigo (Terraform). Pipelines CI/CD. Monitoramento e observabilidade. Provedores de nuvem (AWS, Azure, GCP). Servicos gerenciados.

2. CONCEITOS FUNDAMENTAIS

A seguir, apresentamos os conceitos-chave desta disciplina com explicacoes didaticas e exemplos praticos para facilitar o aprendizado.

2.1 Containers e Docker

Containers sao ambientes isolados e leves que empacotam uma aplicacao com todas as suas dependencias. Docker e a plataforma mais popular para criar e gerenciar containers. Diferente de maquinas virtuais, containers compartilham o kernel do sistema operacional, sendo muito mais eficientes. Um Dockerfile define como construir a imagem, e o Docker Compose orquestra multiplos containers.

Exemplo:

```
Dockerfile para aplicacao Python:
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "app.py"]

Comandos basicos:
docker build -t minha-app .
docker run -p 8000:8000 minha-app
docker-compose up -d
```

2.2 CI/CD e Pipelines de Entrega

CI (Integracao Continua) automatiza a compilacao e testes a cada commit. CD (Entrega Continua) automatiza o deploy em ambientes de staging e producao. Pipelines CI/CD garantem que o codigo e testado e implantado de forma consistente e rapida. Ferramentas populares incluem GitHub Actions, Jenkins e GitLab CI.

Exemplo:

```
Exemplo de GitHub Actions (.github/workflows/ci.yml):
name: CI Pipeline
on: [push, pull_request]
jobs:
test:
runs-on: ubuntu-latest
steps:
```

```
- uses: actions/checkout@v3
- uses: actions/setup-python@v4
- run: pip install -r requirements.txt
- run: pytest tests/
```

2.3 Infraestrutura como Codigo (IaC)

IaC permite definir e gerenciar infraestrutura usando arquivos de configuracao em vez de processos manuais. Terraform e a ferramenta mais popular, suportando multiplos provedores de nuvem. CloudFormation e a opcao nativa da AWS. IaC traz versionamento, reproducibilidade e automacao para a infraestrutura.

Exemplo:

```
Exemplo Terraform (main.tf):
resource "aws_instance" "web" {
ami = "ami-0c55b159cbfafe1f0"
instance_type = "t2.micro"
tags = {
Name = "WebServer"
}
}

terraform init
terraform plan
terraform apply
```

3. RESUMO DOS PRINCIPAIS TOPICOS

O conteudo programatico da disciplina esta organizado nas seguintes unidades:

- Unidade 1: Cultura DevOps - principios, praticas e ferramentas
- Unidade 2: Containers com Docker - imagens, volumes, redes
- Unidade 3: Orquestracao com Kubernetes - pods, services, deployments
- Unidade 4: Infraestrutura como Codigo - Terraform e CloudFormation
- Unidade 5: Pipelines CI/CD - GitHub Actions, Jenkins, GitLab CI
- Unidade 6: Monitoramento e observabilidade - Prometheus, Grafana, ELK
- Unidade 7: AWS - EC2, S3, Lambda, RDS, ECS, EKS
- Unidade 8: Arquiteturas cloud-native e serverless

Cada unidade aborda um aspecto fundamental da disciplina, construindo progressivamente o conhecimento necessario para dominar o tema. Recomenda-se estudar as unidades na ordem apresentada.

4. EXERCICIOS DE FIXACAO

Resolva os exercicios abaixo para verificar seu entendimento dos conceitos apresentados.

Questao 1 (Multipla Escolha)

Qual a principal diferenca entre containers e maquinas virtuais?

a) Containers sao mais seguros
b) Containers compartilham o kernel do SO host
c) VMs sao mais rapidas
d) Containers precisam de mais memoria

Questao 2 (Dissertativa)

Explique o conceito de CI/CD e como ele melhora o processo de desenvolvimento de software.

Questao 3 (Multipla Escolha)

Qual ferramenta e usada para orquestracao de containers em larga escala?

a) Docker Compose
b) Terraform
c) Kubernetes
d) Ansible

Questao 4 (Dissertativa)

Descreva as vantagens de usar Infraestrutura como Codigo (IaC) em comparacao com configuracao manual.

Questao 5 (Multipla Escolha)

O que e um Dockerfile?

a) Um container em execucao
b) Um arquivo que define como construir uma imagem Docker
c) Um servico de orquestracao
d) Um arquivo de configuracao do Kubernetes

5. DICAS DE ESTUDO E RECURSOS ADICIONAIS

Para maximizar seu aprendizado nesta disciplina, siga estas recomendacoes:

- Instale Docker e pratique criando containers para seus projetos.
- Crie pipelines CI/CD para seus repositorios no GitHub.
- Explore o free tier da AWS para praticar servicos de nuvem.
- Estude para certificacoes como AWS Cloud Practitioner ou Docker Certified Associate.
- Leia o livro "O Projeto Phoenix" para entender a cultura DevOps.

Bibliografia Recomendada:

- KIM, G. et al. O Projeto Phoenix. Alta Books, 2016.
- MORRIS, K. Infrastructure as Code. O'Reilly, 2020.

• BURNS, B. Kubernetes: Tudo Sobre Orquestracao de Containers. Novatec, 2019.

6. GLOSSARIO DE TERMOS IMPORTANTES

Consulte este glossario sempre que encontrar um termo desconhecido durante seus estudos.

- **Container**: ambiente isolado e leve para executar aplicacoes.
- **Docker**: plataforma para criar, distribuir e executar containers.
- **Kubernetes**: sistema de orquestracao de containers em larga escala.
- **CI/CD**: Integracao Continua / Entrega Continua.
- **Terraform**: ferramenta de Infraestrutura como Codigo (IaC).
- **Pipeline**: sequencia automatizada de etapas (build, test, deploy).
- **Serverless**: modelo onde o provedor gerencia a infraestrutura automaticamente.
- **Observabilidade**: capacidade de entender o estado interno de um sistema.
- **IaC**: Infrastructure as Code — infraestrutura definida em arquivos de configuracao.